

Software Engineering II

Semester 2026-I

Final Project Definition and Delivery

Eng. Liliana Marcela Olarte, M.Sc.
Eng. Carlos Andrés Sierra, M.Sc.

Lecturers
Computer Engineering Program
School of Engineering
Universidad Nacional de Colombia

Congratulations on reaching the final stage of your *Software Engineering II* course project! This document consolidates the requirements from **Workshops 1-4** into a comprehensive final project delivery. Your submission should demonstrate a complete, well-documented, and fully functional software system with a **react-native** mobile frontend, robust backend services in Java (**Spring Boot**) and Python (**FastAPI**), comprehensive API documentation, professional testing, containerization, and CI/CD deployment pipelines.

Final Project Scope and Objectives:

- **Requirements Engineering (Workshop 1):** Define functional and non-functional requirements, create user stories with acceptance criteria, and develop a user story map for prioritization.
- **System Design (Workshop 2):** Produce design artifacts including CRC cards, wireframes/mockups, business process diagrams, architecture diagrams, UML class diagrams, and relational database models.
- **Full Stack Implementation (Workshop 3):** Implement a relational database using PostgreSQL or MySQL, backend services in Java (**Spring Boot** with JPA/Hibernate) and Python (**FastAPI** or **Flask** with SQLAlchemy), a mobile frontend using **react-native**, and comprehensive **Swagger/OpenAPI** API documentation with interactive Swagger UI. Include unit tests using **JUnit** (Java) and **pytest** (Python).

Carlos Andrés Sierra, Computer Engineer, M.Sc. in Computer Engineering, Lecturer at Universidad Nacional de Colombia.

Any comment or concern regarding this project can be sent to Carlos A. Sierra at: casierrav@unal.edu.co.

- **Testing, Deployment, and MVP Delivery (Workshop 4):** Provide comprehensive unit tests, integration tests validating ORM layers, acceptance tests for key user stories, containerization using `Docker` and `Docker Compose`, CI/CD pipeline with `GitHub Actions`, and a complete, deployable MVP.

Methodology and Deliverables:

1. Workshop 1 – Requirements and Planning:

- Provide functional and non-functional requirements documentation.
- Deliver user stories with acceptance criteria in BDD format (Given-When-Then).
- Include a user story map for planning and prioritization.

2. Workshop 2 – System Design:

- Provide CRC cards for main classes and their responsibilities.
- Include mockups or wireframes for key application screens.
- Document business processes using activity diagrams or BPMN.
- Deliver architecture diagrams showing all components and interactions.
- Provide UML class diagrams with attributes, methods, and relationships.
- Include relational database ER diagrams and schema design.

3. Workshop 3 – Full Stack Implementation:

- Deliver SQL scripts and migration files for `PostgreSQL` or `MySQL` database with sample data.
- Provide Java backend using `Spring Boot` with `JPA/Hibernate` ORM.
- Provide Python backend using `FastAPI` or `Flask` with `SQLAlchemy` ORM.
- Deliver comprehensive `Swagger/OpenAPI` API documentation with interactive `Swagger UI`.
- Implement mobile frontend using `react-native`.
- Include unit tests for both backends (`JUnit` for Java, `pytest` for Python).
- Provide integration evidence (screenshots, demos, code snippets).

4. Workshop 4 – Testing, Deployment, and MVP:

- Provide comprehensive unit tests with coverage reports.
- Implement integration tests validating ORM layers.
- Develop acceptance tests for key user stories.
- Create `Dockerfile` for all major components and `docker-compose.yml` for orchestration.
- Implement `GitHub Actions` CI/CD workflow with automated testing, building, and deployment.

- Deliver a fully functional MVP with all components working together.
- Provide deployment documentation and evidence of successful pipeline runs.

5. Final Project Repository:

- Organize all deliverables in a folder named **Final-Project** in your course repository.
- Include subfolders for each workshop (**Workshop-1**, **Workshop-2**, **Workshop-3**, **Workshop-4**).
- Provide a master **README.md** explaining the entire project structure, setup instructions, and links to all documentation.
- Compile all sections into a single comprehensive PDF document.

6. Demonstration (Mandatory):

- Prepare and deliver a brief presentation or video (5-10 minutes) demonstrating your system's main features, API endpoints, **react-native** frontend, comprehensive API documentation via Swagger UI, and CI/CD deployment pipeline.
- The demo is a required part of the final evaluation.

Project Requirements Checklist:

- Clear and complete requirements (functional and non-functional).
- Well-defined user stories and acceptance criteria.
- User story mapping for planning.
- CRC cards, mockups, business processes, architecture diagram, class diagrams.
- Relational database schema with proper relationships and normalization.
- Java backend (**Spring Boot**) with **JPA/Hibernate** ORM.
- Python backend (**FastAPI/Flask**) with **SQLAlchemy** ORM.
- Comprehensive **Swagger/OpenAPI** API documentation with interactive Swagger UI.
- Mobile frontend using **react-native**.
- Unit tests for both backends (**JUnit**, **pytest**) with coverage reports.
- Integration tests validating ORM layers and backend services.
- Acceptance tests for key user stories.
- **Docker** and **Docker Compose** files for all services.
- GitHub Actions CI/CD pipeline with automated testing, building, and deployment.
- Fully functional MVP demonstrating all features.

- Organized and comprehensive documentation.

Recommended Technology Stack:

- **Database:** Relational database using PostgreSQL or MySQL with proper schema design, relationships, and normalization.
- **Java Backend:** Spring Boot framework with JPA/Hibernate ORM, JUnit for unit testing, and Swagger/Springdoc for API documentation.
- **Python Backend:** FastAPI or Flask framework with SQLAlchemy ORM, pytest for unit testing, and Swagger/OpenAPI for API documentation.
- **Mobile Frontend:** react-native for cross-platform mobile application development.
- **API Communication:** REST principles with JSON data format, Swagger/OpenAPI specifications.
- **Integration Testing:** Testcontainers for database integration tests.
- **Containerization:** Docker, Docker Compose, Dockerfile for all services.
- **CI/CD:** GitHub Actions for automated testing, building, and deployment.

Examples of Application Ideas:

- Task manager or to-do list application.
- Simple blog or content management system.
- Basic inventory or product catalog.
- Contact management or address book.
- Event registration or booking system.

Deadline: Thursday, July 23rd, 2026, at 18:00. Late submissions may affect your grading according to course policies.

Notes:

- All documents must be in **English**.
- Ensure comprehensive test coverage including ORM layers (JPA/Hibernate for Java, SQLAlchemy for Python).
- Verify that Swagger/OpenAPI documentation is automatically generated and accessible via interactive Swagger UI.
- Test your Docker containers and docker-compose orchestration locally before final submission.

- Ensure your GitHub Actions CI/CD pipeline demonstrates successful automated testing, building, and deployment.
- Cite any references (articles, tutorials, frameworks) that influenced your design choices.
- Focus on clarity, completeness, coherence across all workshops, and professional presentation.
- This final project is the culmination of your learning across all four workshops. It should demonstrate your mastery of requirements engineering, system design, full-stack implementation, and DevOps practices.

Good luck! Your final project showcases your ability to design, implement, test, and deploy a professional-grade software system using modern software engineering practices, industry-standard technologies, and DevOps principles. Make it exceptional!